SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – SCENARIO BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Scenario Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**M.MANASA**	**Reg. No.**	**192425189**
Section/Batch	**Slot D / 2024**	**Date**	**31-10-2026**

Code of Conduct:

I,_M.Manasa(192425189)_, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

m.manasa

ASSIGNMENT QUESTION

Analyze using a case study the fundamental concepts of website development, HTML5 document structure, semantic elements and the important features introduced in HTML5.

1. Introduction

Website development is the process of creating websites that people can access through a web browser. It includes planning the content, designing the interface, writing code, testing the pages and publishing the website on a web server. HTML provides the structure, CSS controls presentation, and JavaScript adds interactive behaviour. HTML5 introduced improved semantic elements and built-in support for multimedia, graphics, forms and modern web applications.

2. Case Study: College Event Website

Consider a college that wants a website to announce technical events, display schedules, share photos and allow students to register. The website should be easy to navigate, work on phones and laptops, and provide accessible, well-organized content. HTML5 can be used to build the page structure, CSS can make the layout responsive, JavaScript can validate registration details, and a server-side application can store registrations.

How the website works

1. A student opens the college event website in a browser.
2. The browser requests the page from the web server using HTTP or HTTPS.
3. The server sends the HTML document and related CSS, JavaScript, images or media files.
4. The browser parses the HTML, builds the page structure and displays the content.
5. When the student submits a registration form, the browser validates the fields and sends the information to the server for processing.

Goals and target users

Before any code is written, the development team identifies who will use the website and what the site must achieve. For the college event website the main goals are to publish accurate event information, reduce manual paperwork by collecting registrations online, and present a consistent image of the institution.

- **Students:** First-year and senior students who want to browse events and register quickly, usually from a mobile phone.
- **Organizers:** Event coordinators who need to publish schedules, update details and view the list of registered participants.
- **Visitors and guests:** Faculty, parents and visiting participants who need venue, timing and contact information.

Website development life cycle

A website is normally built in a sequence of stages. Each stage produces something that the next stage uses, and the process is repeated whenever the site needs improvement.

- **Planning:** Gather requirements, list the pages needed (home, events, schedule, gallery, registration, contact) and decide what each page must contain.
- **Design:** Sketch wireframes, define the navigation structure, and choose colours, fonts and image style.
- **Development:** Write semantic HTML5 for structure, CSS for layout and appearance, and JavaScript for behaviour; add server-side code for storing registrations.
- **Testing:** Check links, forms, responsiveness, accessibility, loading speed and compatibility across browsers and devices.
- **Deployment:** Upload the site to a hosting server, connect a domain name and enable HTTPS.
- **Maintenance:** Fix errors, update event details, back up data and review security regularly.

3. Fundamental Concepts of Website Development

- **HTML (HyperText Markup Language):** Defines the structure and meaning of page content, such as headings, paragraphs, images, links and forms.
- **CSS (Cascading Style Sheets):** Controls colours, fonts, spacing, layout and responsive design for different screen sizes.
- **JavaScript:** Adds interaction, such as menu controls, form validation and dynamically updating page content.
- **Web browser and web server:** The browser requests and displays pages; the server stores or generates resources and sends responses.
- **HTTP/HTTPS:** Protocols used for browser-server communication. HTTPS protects data in transit using encryption.

- **Responsive design and accessibility:** Responsive layouts adapt to different devices. Semantic HTML, labels and meaningful alternative text help people use the website with different abilities and tools.
- **Testing and deployment:** Developers test links, forms, display and compatibility before publishing the site on a hosting server.
- **Domain names and hosting:** A domain name (for example, techfest.college.edu) is a readable address that the Domain Name System (DNS) translates into the IP address of the server hosting the site.
- **Front end and back end:** Front-end code runs in the browser (HTML, CSS, JavaScript) and controls what the visitor sees. Back-end code runs on the server and handles tasks such as saving registrations in a database and sending confirmation emails.
- **Performance:** Pages should load quickly. Compressing images, minimising CSS and JavaScript files and avoiding unnecessary scripts improve the experience, particularly on slow mobile connections.
- **Search engine optimization (SEO):** Using clear headings, descriptive page titles, meta descriptions and meaningful link text helps search engines understand the site and helps visitors find it.

4. Basic HTML5 Document Structure

A standard HTML5 document includes a doctype, a root html element, a head section for metadata and a body section for visible content.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Event Website</title>
</head>
<body>
  <header>
    <h1>College Tech Fest</h1>
  </header>
  <nav>
    <a href="#events">Events</a>
    <a href="#register">Register</a>
  </nav>
  <main>
```

```
    <section id="events">
      <h2>Upcoming Events</h2>
      <p>Explore workshops and competitions.</p>
    </section>
    <section id="register">
      <h2>Student Registration</h2>
      <form>
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <button type="submit">Register</button>
      </form>
    </section>
  </main>
  <footer>
    <p>College Tech Fest</p>
  </footer>
</body>
</html>
```

Explanation of the structure

- **<!DOCTYPE html>** Declares that the document uses modern HTML.
- **<html lang="en">** Root element; the language attribute identifies the page language.
- **<head>** Contains metadata, the page title and links to resources such as CSS.
- **<meta charset="UTF-8">** Sets the character encoding so text is interpreted correctly.
- **<meta name="viewport" ...>** Helps the page display at an appropriate width on mobile devices.
- **<title>** Sets the title shown in the browser tab and often used in bookmarks.
- **<body>** Contains the content displayed on the page.

Other commonly used head and body elements

- **<meta name="description" content="...">** Provides a short summary of the page that search engines may display in results.
- **<link>** Connects an external CSS file to the page, for example <link rel="stylesheet" href="style.css">.
- **<script>** Loads JavaScript. Placing <script src="script.js" defer></script> lets the page be parsed before the script runs.
- **Headings:** Headings <h1> to <h6> create a content hierarchy. A page should normally have one <h1>, with <h2> and <h3> used for sub-topics in order.

- **Images:** <img src="..." alt="..."> displays an image; the alt text describes it for screen-reader users and appears if the image cannot load.
- **Lists and tables:** Lists (<ul>, <ol>, <li>) and tables (<table>, <tr>, <th>, <td>) organize related items and tabular data such as an event timetable.

Example: linking CSS and JavaScript

The following head and body fragments show how an external stylesheet and script are attached to the event page, and how the viewport meta tag works with a simple responsive rule.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Events, schedule and registration for College Tech Fest">
  <title>College Event Website</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>

/* style.css */
nav a { margin-right: 1rem; }
section { padding: 1rem; }
@media (max-width: 600px) {
  nav a { display: block; margin: 0.5rem 0; }
}
```

On a screen narrower than 600 pixels the navigation links stack vertically, which makes them easier to tap on a phone.

5. HTML5 Semantic Elements

Semantic elements describe the purpose of the content they contain. They make the page easier for developers to understand and can help assistive technologies identify important page regions.

- **<header>** Introductory content, a logo or the page heading.
- **<nav>** A group of important navigation links.
- **<main>** The main, unique content of the page.
- **<section>** A related group of content, usually with a heading.
- **<article>** A self-contained item such as a news story or event announcement.
- **<aside>** Related or supplementary information, such as a sidebar.
- **<footer>** Footer information such as contact details or copyright.
- **<figure> and <figcaption>** Media or illustrations with an optional caption.

In the case study, <header> contains the event website title, <nav> provides links to event details, <main> holds the main content, <section> groups event information and registration, and <footer> provides the closing information. These elements are more meaningful than using for every part of the page.

Additional semantic and text-level elements

- **<time>** Represents a date or time in a machine-readable form, for example <time datetime="2026-12-10">10 December 2026</time>.
- **<mark>** Highlights text that is relevant in the current context, such as a search term or an important notice.
- **<address>** Provides contact information for the page or article author, such as the event coordinator.
- **<details> and <summary>** Creates a collapsible block of content that the visitor can open or close without JavaScript, useful for FAQs.
- **<progress> and <meter>** Shows the completion of a task, such as the progress of registration steps, and a measurement within a known range, such as available seats.

Why semantic elements matter

- **Accessibility:** Screen readers can list the landmarks of a page (header, navigation, main, footer) and let users jump directly to the section they need.
- **Search engine optimization:** Search engines can better identify the main content, headings and navigation of a page.
- **Maintainability:** Meaningful tag names make the code easier to read, update and share among team members than a page made only of elements.
- **Consistent styling:** Consistent semantic structure makes it easier to apply styles and scripts to the correct regions of the page.

Example: event announcements using article, aside and figure

```
<section id="events">
  <h2>Upcoming Events</h2>
  <article>
    <h3>Web Design Workshop</h3>
    <p>Date: <time datetime="2026-12-10">10 December 2026</time></p>
    <p>Learn to build a responsive page using HTML5 and CSS.</p>
    <figure>
      <img src="workshop.jpg" alt="Students working on laptops during a workshop">
      <figcaption>Last year's workshop</figcaption>
```

```
    </figure>
  </article>
  <aside>
    <h3>Important Notice</h3>
    <p>Registration closes on <mark>5 December</mark>.</p>
  </aside>
</section>
```

Here each event is an independent <article>, the notice is related but secondary information placed in an <aside>, and the photograph and its caption are grouped with <figure> and <figcaption>.

6. Important Features Introduced or Improved in HTML5

- **Audio and video:** The <audio> and <video> elements allow media playback without requiring a separate plug-in.
- **Canvas graphics:** The <canvas> element provides a drawing area that JavaScript can use for charts, animations and simple games.
- **Scalable Vector Graphics (SVG):** SVG supports resolution-independent graphics and icons in web pages.
- **Improved forms:** Input types such as email, date, number and url, along with attributes such as required and placeholder, support more useful forms and basic browser validation.
- **Local storage and session storage:** Web Storage lets a site store small amounts of data in the browser. Local storage persists across browser sessions; session storage is limited to a tab's session.
- **Geolocation API:** With user permission, a website can request the device's location for location-based features.
- **Drag and drop:** HTML drag-and-drop APIs support moving items within a page when implemented with appropriate scripting.
- **Web workers:** Web Workers can run certain JavaScript tasks in the background without blocking the main page interface.
- **Semantic structure:** Elements such as <header>, <nav>, <main>, <section> and <footer> describe page regions clearly.
- **Mobile-friendly support:** The viewport metadata and CSS media queries help developers create layouts that adapt to phones, tablets and desktops.
- **More form controls:** Additional input types include tel, time, range, color, search and datetime-local. Attributes such as pattern, min, max, maxlength, autofocus and autocomplete give further

control, and the <datalist> element offers suggestions while typing. Mobile devices display a suitable keyboard for each type.

- **New interface elements:** The <progress> and <meter> elements display task progress and measured values, and the <output> element shows the result of a calculation.
- **WebSockets:** WebSockets provide a continuous two-way connection between browser and server, useful for live updates such as announcing results during an event.
- **Client-side databases and offline support:** IndexedDB is a browser database for storing larger amounts of structured data than Web Storage allows. Service workers and the Cache API support offline use of web applications.
- **Media captions:** The <track> element adds subtitles and captions to <video> and <audio>, improving accessibility.
- **Custom data attributes:** Custom data-* attributes allow extra information to be stored on elements and used by JavaScript, for example <li data-event-id="12">.
- **Communication with the server:** The Fetch API and related JavaScript interfaces let a page request data from the server without reloading, so parts of the page can be updated dynamically.

Example: audio, video and canvas

```
<video controls width="480" poster="intro.jpg">
   <source src="intro.mp4" type="video/mp4">
   <source src="intro.webm" type="video/webm">
   <track src="captions.vtt" kind="captions" srclang="en" label="English">
   Your browser does not support the video element.
</video>

<canvas id="seats" width="300" height="100"></canvas>
<script>
   const ctx = document.getElementById("seats").getContext("2d");
   ctx.fillStyle = "#2e74b5";
   ctx.fillRect(10, 10, 180, 40);   // seats booked
   ctx.strokeRect(10, 10, 280, 40); // total seats
</script>
```

The <source> elements let the browser choose a format it supports, the text inside <video> is shown only when the element is not supported, and the canvas script draws a simple bar showing how many seats are booked.

Example: Web Storage

```
// Remember that the visitor closed the notice
localStorage.setItem("noticeClosed", "yes");
```

```
// Read it back on the next visit
if (localStorage.getItem("noticeClosed") === "yes") {
  document.querySelector("aside").hidden = true;
}
```

7. Applying HTML5 Features to the Case Study

1. Use semantic elements to organize event announcements, schedules and the registration area.
2. Use the email input type and required attribute to check basic registration details in the browser.
3. Use video to show a short event introduction and figure/figcaption to describe event photographs.
4. Use responsive CSS and the viewport setting so students can use the site on mobile devices.
5. Use local storage only for suitable non-sensitive preferences, such as a dismissed notice; send actual registration data securely to the server.
6. Use accessible labels, keyboard-friendly controls and meaningful link text so more students can navigate the site.
7. Use the details and summary elements to present frequently asked questions about eligibility, fees and venue.
8. Use the time element for event dates and the address element for the coordinator's contact details.
9. Use the track element to add captions to the introduction video so that all students can follow it.
10. Test the website on phones, tablets and several browsers before publication, and validate the HTML.

Extended registration form

The basic form in Section 4 can be extended with additional HTML5 input types and attributes so that the browser checks more of the details before submission.

```
<form action="/register" method="post">
  <label for="name">Full name:</label>
  <input type="text" id="name" name="name" required autofocus>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <label for="phone">Mobile number:</label>
  <input type="tel" id="phone" name="phone" pattern="[0-9]{10}"
      placeholder="10-digit number" required>

  <label for="event">Event:</label>
  <select id="event" name="event">
    <option>Web Design Workshop</option>
```

```
      <option>Coding Contest</option>
    </select>

    <label for="date">Preferred date:</label>
    <input type="date" id="date" name="date" min="2026-12-01">

    <button type="submit">Register</button>
</form>
```

The required attribute prevents empty submission, the email type checks the address format, the pattern attribute limits the mobile number to ten digits, and the min attribute prevents selection of an earlier date. The server must still check all of these values again.

Comparison: earlier HTML and HTML5

Aspect	Earlier HTML practice	HTML5 approach
Page structure	Many elements with ids such as "header" and "nav"	Semantic elements: <header>, <nav>, <main>, <section>, <footer>
Doctype	Long, complex doctype declarations	Short <!DOCTYPE html>
Multimedia	Plug-ins such as Flash or Java applets	Native <audio> and <video> elements
Graphics	Images or plug-ins	<canvas> and inline SVG
Forms	Mostly text fields checked with JavaScript	New input types, required, pattern and built-in validation
Client data storage	Cookies only	Web Storage and IndexedDB
Mobile support	Separate mobile sites	Viewport meta tag and responsive layouts

8. Benefits and Limitations

Benefits:

- Provides a clear and maintainable page structure.
- Supports audio, video, graphics and richer forms in modern browsers.

- Improves meaning and navigation through semantic elements.
- Helps developers create responsive and accessible experiences.
- Reduces dependence on third-party plug-ins, which lowers security risks and improves compatibility.
- Works across desktops, tablets and phones, which reduces the cost of building separate sites.
- Is an open standard maintained by the WHATWG and supported by all major browsers.

Limitations and precautions:

- HTML alone cannot create a complete data-driven website; CSS, JavaScript and often server-side code are also needed.
- Browser support and behaviour can vary, so features should be tested across target browsers.
- Client-side form validation improves usability but is not a security boundary; the server must validate submitted data too.
- Location access and browser storage should be used transparently and with appropriate privacy safeguards.
- Older browsers may not support some newer features, so fallback content or alternatives may be needed.
- Large audio and video files can slow down a page unless they are compressed and loaded carefully.
- Overusing JavaScript and heavy media can reduce performance on low-cost devices.

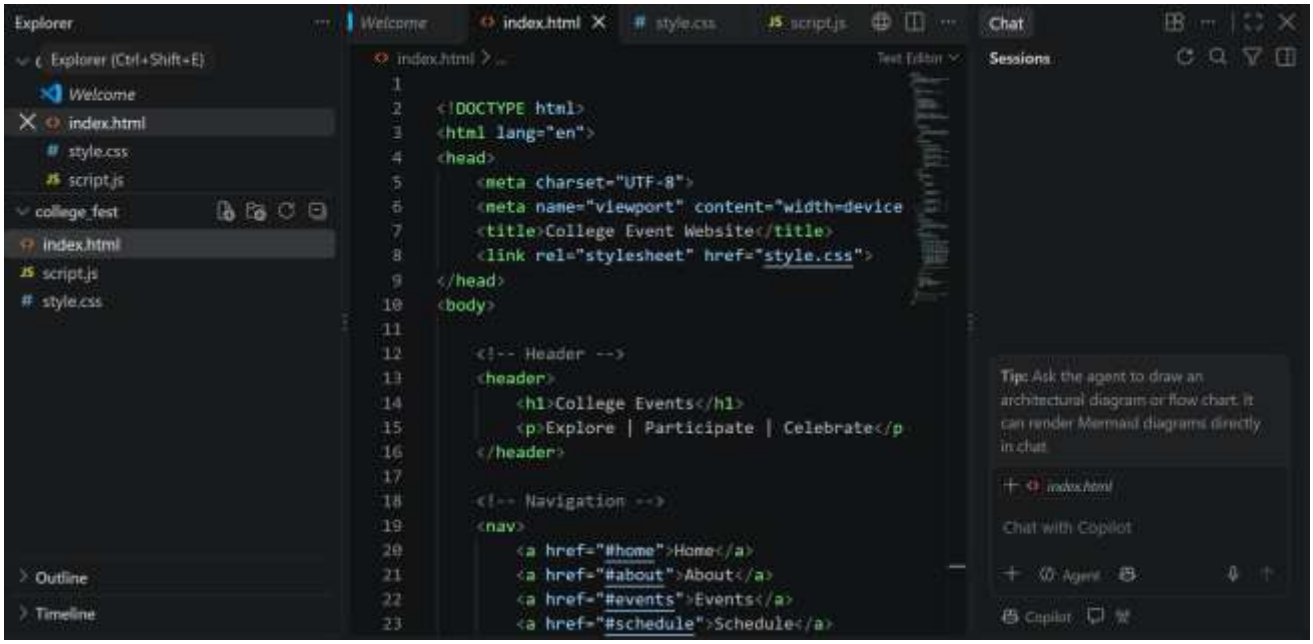
9. Best Practices for HTML5 Development

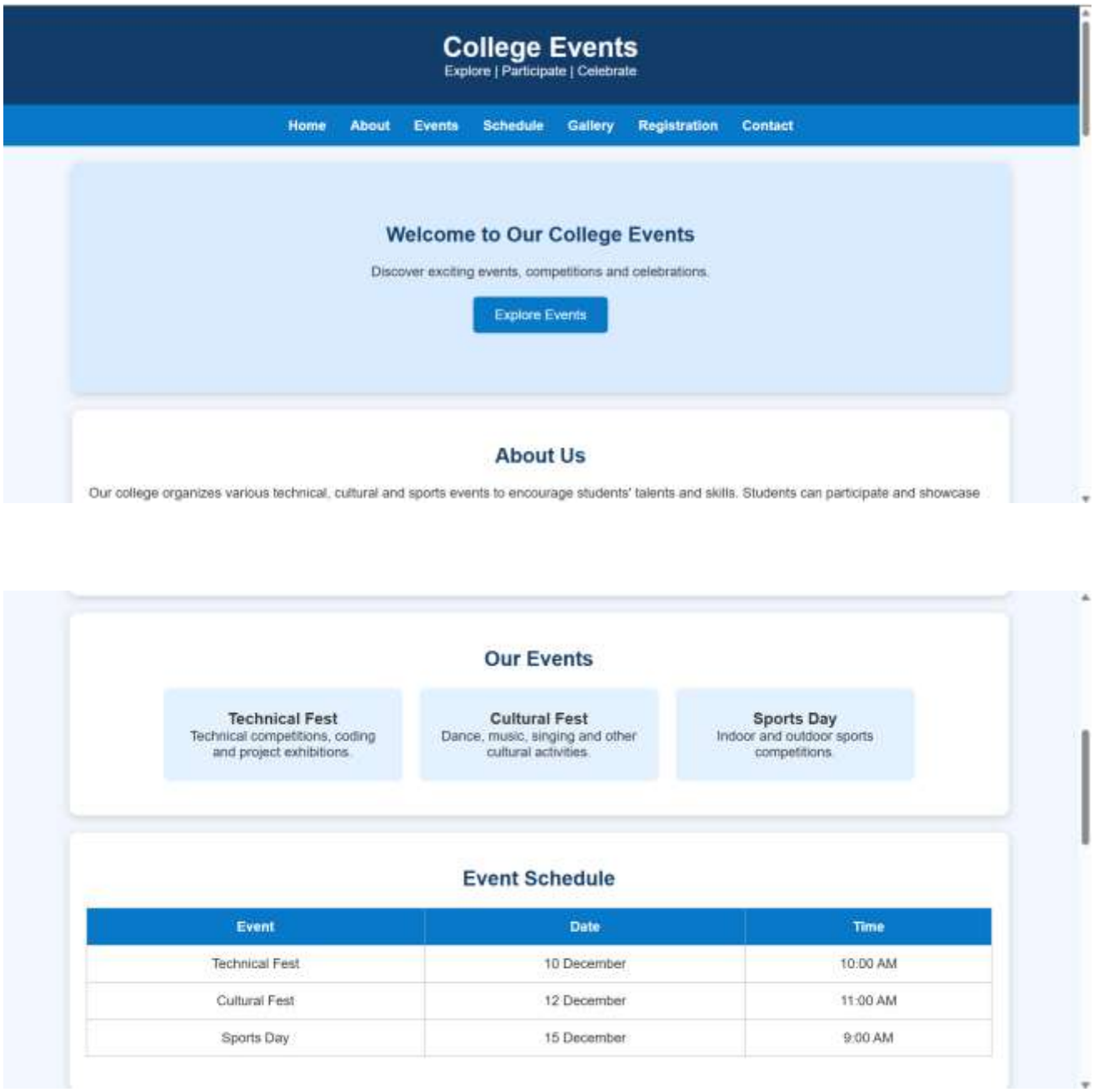
- Use the correct semantic element for each part of the page instead of generic elements.
- Include <!DOCTYPE html>, the lang attribute, the charset and the viewport meta tag in every page.
- Write meaningful alt text for images and connect every form control with a <label>.
- Keep structure (HTML), presentation (CSS) and behaviour (JavaScript) in separate files.
- Use one <h1> per page and keep headings in a logical order.
- Validate the markup with the W3C Markup Validation Service and test with a keyboard and a screen reader.
- Use HTTPS, validate and sanitize all input on the server, and avoid storing sensitive data in browser storage.

- Optimize images and media, and test the site on different browsers and screen sizes before deployment.

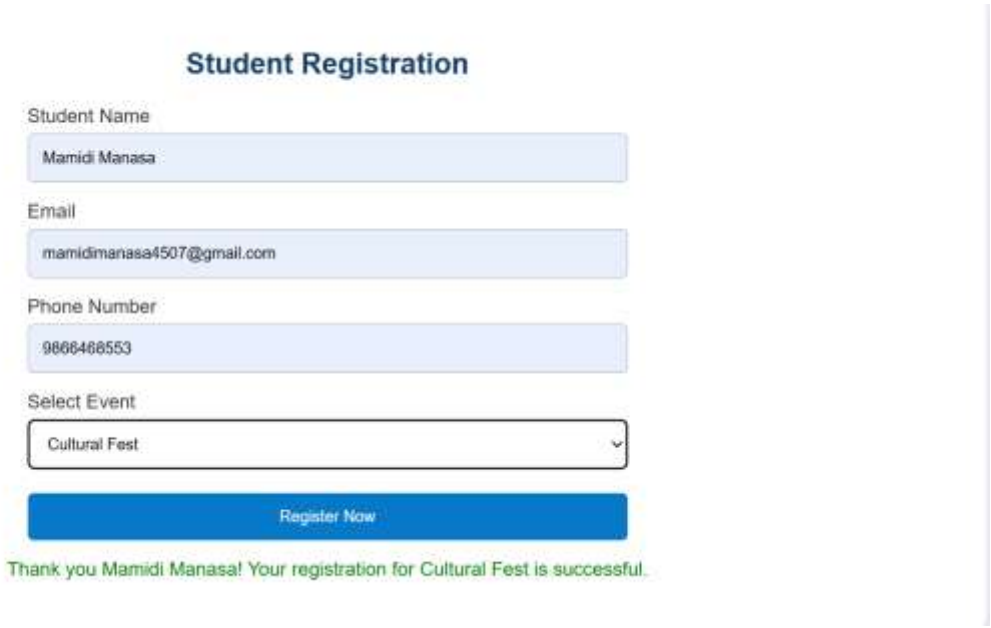
10. Conclusion

The college event website case study shows how the fundamentals of website development work together. HTML5 provides the document structure, semantic elements give content meaningful organization, CSS controls presentation, and JavaScript supports interaction. HTML5 features such as multimedia, improved forms, canvas, browser storage and web APIs help developers build useful modern websites. Good testing, accessibility, responsive design and secure server-side validation are also essential for a reliable website. By following the website development life cycle and the best practices described above, a developer can deliver a college event website that is easy to maintain, accessible to all students and ready for future improvement.




College Events
Explore | Participate | Celebrate
Home
About
Events
Schedule
Gallery
Registration
Contact
Welcome to Our College Events
Discover exciting events, competitions and celebrations.
Explore Events
About Us
Our college organizes various technical, cultural and sports events to encourage students' talents and skills. Students can participate and showcase

Our Events
Technical Fest
Technical competitions, coding and project exhibitions
Cultural Fest
Dance, music, singing and other cultural activities
Sports Day
Indoor and outdoor sports competitions
Event Schedule
Event
Date
Time
Technical Fest
10 December
10:00 AM
Cultural Fest
12 December
11:00 AM
Sports Day
15 December
9:00 AM



Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Website development concepts	20%	Explains all key concepts clearly and correctly.	Explains most concepts correctly.	Explains a few concepts; some unclear.	Concepts missing or incorrect.	20
Case study	20%	Case study is clear and fully linked to HTML5.	Case study is clear with some links to HTML5.	Case study is general and weakly linked.	Case study missing.	20
HTML5 structure and semantic elements	25%	Correct structure and semantic elements with examples and clear explanation.	Mostly correct with some examples.	Incomplete or has errors.	Missing or incorrect.	25
HTML5 features	25%	Covers many features, each explained with a use.	Covers most features with brief explanations.	Covers few features with little explanation.	Features missing or incorrect.	25
Presentation and references	10%	Well organized, error-free, with references.	Mostly organized with minor errors.	Uneven organization; several errors.	Poorly organized; no references.	10